

Daily Paper Review: SkillsVote — Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution

Internbot Paper Review

May 20, 2026

1 Paper Summary

1.1 Problem and Motivation

Long-horizon LLM agents produce execution trajectories containing reusable operational experience, but raw trajectories are a weak substrate for long-term reuse: they are lengthy, noisy, and tightly bound to local environments. Agent Skills—structured packages of procedural instructions, scripts, templates, and applicability conditions—provide a more compact representation. However, at ecosystem scale, the challenge shifts from authoring individual skills to **governing a continuously expanding library**. Liu et al. [1] identify four failure modes: (1) indiscriminate exposure of weakly related skills can *degrade* agent performance; (2) no attribution layer exists to determine whether success stemmed from the skill, the agent’s exploration, or environmental factors; (3) a granularity mismatch exists between task-level success signals (too coarse) and individual tool calls (too fragmented); and (4) current discovery signals (popularity, metadata) are insufficient for establishing execution readiness.

1.2 Proposed Method: SkillsVote

SkillsVote introduces a **lifecycle governance framework** connecting four coupled processes into a closed loop: collection and profiling, recommendation, attribution, and controlled evolution. The “vote” metaphor operates bidirectionally: skills vote into agent context (recommendation gates which skills are exposed), and evidence votes into the library (attribution gates which experience modifies persistent skills).

Collection and Profiling. A million-scale corpus is built from GitHub SKILL.md files, where each skill is a directory-level package containing instructions plus optional scripts, references, and assets. Skills are profiled along three dimensions: *runtime requirements* (OS, permissions, APIs), *quality* (consistency, completeness, task orientation), and *verifiability* (low-ambiguity success conditions, reproducible sandboxes). For verifiable skills, tasks are synthesized in the Harbor format with executable verifiers.

Agentic Skill Recommendation. Before the solver agent executes, a separate recommendation agent searches the skill library using filesystem-native tools (Glob, Grep, Read)—*not* embedding-based retrieval. It selectively reads candidate SKILL.md files, selects skills that cover the task and fit the target environment, and produces a compact set of exposed skills plus a usage guide. This record anchors later attribution by tracking which skills were actually exposed.

Subtask-Level Attribution. The key technical contribution. SkillsVote inserts an attribution layer between full trajectories and individual tool calls. A **subtask** is defined as the smallest semantically complete unit with one standalone objective, one primary evaluation signal, and at most one associated skill. Attribution operates along three axes:

- **Outcome evidence** (3 types): **environment** (observable feedback), **human** (preference-dependent), **unknown** (no signal).
- **Responsibility assignment** (11 categories): 3 successful (skill viewed but unused, no skill seen, skill used with extra exploration), 5 failure (skill issue, agent limit, client env, external env, unknown env), and 3 uncertain (human judge required, inconclusive, no judge). Only the 3 success categories can trigger evolution.
- **Reusable delta**: For skill-linked subtasks, localizes the specific skill knowledge spans actually relied upon (via file path, line ranges, capability, usage) and extracts only reusable discoveries while discarding task-specific constants.

Attribution is implemented by *resuming the original agent session* rather than compressing it, preserving full contextual detail including encrypted chain-of-thought.

Controlled Skill Evolution. Admissible units must be both successful and contain reusable exploration. Aggregated evidence is routed to five action types: **error_fix**, **knowledge_addition**, **prerequisite_addition** (all editing existing skills), **create_skill**, or **skip**. Conservative edit rules assume most of the skill is already correct, prefer local insertions over rewrites, and require all added content to be directly supported by exploration evidence.

1.3 Key Results

Terminal-Bench 2.0 (89 tasks, avg@5). Offline evolution (library frozen from 48 Terminal-Bench Pro tasks) achieves **+7.9 pp** for GPT-5.2 (51.0→58.9) and **+5.8 pp** for GPT-5.4 mini. Online evolution yields smaller gains: **+2.7 pp** and **+1.1 pp** respectively. The offline library’s transfer to unseen tasks is *decoupled from source-task performance*: source accuracy fluctuates non-monotonically while transfer accuracy improves monotonically—proving the library accumulates reusable procedures rather than fitting the source benchmark.

SWE-Bench Pro (731 tasks, avg@1). Online evolution improves both backbones: **+2.6 pp** (GPT-5.2, 47.6→50.2) and **+2.1 pp** (GPT-5.4 mini). Per-repository gains are heterogeneous: nodeb. jumps **+25.0 pp** (47.7→72.7) while some repositories show no improvement or slight regression.

Key Ablation: Recommendation Controls Negative Transfer. On Terminal-Bench 2.0 Hard (30 tasks): without recommendation, online evolution causes net negative transfer (**+3.3 gain / −6.7 loss**). With recommendation, the effect balances (**+6.0 / −6.0**). For offline evolution, recommendation increases positive contribution from **+11.3** to **+15.3 pp** while reducing loss from **−3.3** to **−2.0 pp**. This demonstrates recommendation and evolution play complementary roles: evolution creates reusable knowledge, recommendation decides what to expose.

1.4 Limitations

1. **No learned components:** Unlike SkillOS [2] which trains RL-based curation policies, SkillsVote relies entirely on LLM prompting for all stages. The quality ceiling is bounded by the frozen model’s instruction-following ability.
2. **Online evolution yields modest gains:** **+1.1** to **+2.7 pp** vs. **+5.8** to **+7.9 pp** for offline, suggesting cold-start library building from sequential task streams is inherently difficult.
3. **Single backbone:** Only GPT-5.2 and GPT-5.4 mini via Codex; no evidence for generalization to other model families.

4. **No library scalability analysis:** No formal deduplication or pruning strategy as the library grows.
5. **Attribution quality unvalidated:** No human evaluation or gold-standard annotations for the 11-category attribution taxonomy.
6. **Heavy-tailed effects:** Moderate average gains mask high task-level variance, with skills sometimes causing regressions.

1.5 Relevance to Research Topics

SkillsVote is directly relevant to **Topic 3 (Continual Learning for LLM Agents: lifelong learning, memory)** as it addresses the core challenge of how agents accumulate, curate, and evolve reusable knowledge across tasks. The attribution-gated evolution loop implements a principled form of experience-driven continual improvement without parameter updates, complementing our previous reviews of SkillOS [2] (RL-trained skill curation with insert/update/delete operations), Memanto [3] (typed semantic memory with information-theoretic retrieval), and RubricEM [4] (reflection meta-policy with rubric bank for cross-episode transfer).

2 Follow-up Research Directions

2.1 Direction 1: RL-Trained Attribution for Closed-Loop Skill Governance

Gap. SkillsVote’s attribution relies entirely on LLM prompting with an 11-category taxonomy, producing attribution quality bounded by the frozen model’s instruction-following capability. Our review of SkillOS [2] demonstrated that RL-trained curators with GRPO and composite rewards (task outcome + FC validity + content quality + compression) dramatically outperform prompt-based approaches—an 8B RL curator outperformed Gemini-2.5-Pro as curator. Yet SkillOS lacks the structured attribution taxonomy that SkillsVote introduces. Neither system combines learned curation policies with principled responsibility assignment.

Approach. Train an RL-based attribution policy using SkillsVote’s 11-category taxonomy as the action space and GRPO as the optimization algorithm. The policy takes as input the subtask description, the execution trajectory segment, the linked skill context, and the verifier outcome, and produces an attribution label plus a reusable-delta extraction. The reward function combines: (1) downstream task improvement when the attributed evidence is used for library evolution (measuring whether evolution actually helped on future tasks), (2) attribution consistency across similar subtasks (same root cause should yield same attribution), and (3) evolution efficiency (penalizing spurious library edits that are later overwritten or cause regressions). SkillOS’s grouped task streams provide a natural curriculum: group subtasks by skill and evaluate attribution quality through the resulting evolution’s impact on the next task group.

Feasibility. Medium-high. SkillOS demonstrates GRPO works for curation decisions; SkillsVote provides the structured taxonomy as the action space. The main challenge is defining credit assignment for attribution decisions, since the benefit of correct attribution manifests only when the evolved skill is used on a future task. Starting with offline evaluation (comparing RL-attributed evolution outcomes against prompt-attributed ones on held-out tasks) would establish feasibility before full online training.

2.2 Direction 2: Foresight-Guided Skill Library Compression

Gap. As skill libraries grow through continuous evolution, they face scalability challenges: increasing search space for recommendation and potential redundancy across skills. SkillsVote pro-

vides no formal deduplication or pruning strategy. Our review of Learning to Foresee [5] revealed that on-policy distillation exhibits early low-rank lock-in where top-1% singular components capture 92–95% of update energy. If skill libraries exhibit analogous structural properties—where a small fraction of skills account for most of the library’s utility—then the foresight analysis tools (SVD spectrum, effective rank, directional stability) could identify which skills are structurally redundant.

Approach. Represent the skill library’s utility as a matrix where rows are tasks and columns are skills, with entries being the attribution-measured contribution of each skill to each task. Apply SVD analysis from Learning to Foresee: compute the effective rank, top- k % utility concentration, and identify structurally redundant skills (those whose utility vectors lie in the tail subspace). Skills whose utility is fully captured by the top- k principal skill components can be safely merged or archived without affecting downstream performance. The library evolves with periodic compression checkpoints (analogous to EffOPD’s exponentially spaced triggers), where the compression agent proposes merges validated against a small held-out task set (analogous to EffOPD’s 50-example validation).

Feasibility. Medium. The utility matrix can be constructed from SkillsVote’s attribution records. The main challenge is that skill utility is task-distribution-dependent: a skill redundant for terminal tasks may be essential for SWE tasks. Maintaining domain-specific utility profiles (similar to Memanto’s typed memory categories) would address this.

2.3 Direction 3: Cross-Episode Skill Transfer via Reflection Meta-Policy

Gap. SkillsVote’s offline evolution demonstrates that skill libraries transfer across task distributions (+7.9 pp from Terminal-Bench Pro to Terminal-Bench 2.0), but this transfer is unguided: the frozen library is applied identically regardless of the target domain. Our review of RubricEM [4] showed that a shared-backbone reflection meta-policy with a rubric bank enables both within-episode refinement (exact query match) and cross-episode transfer (semantic similarity retrieval), with Theorem 5 proving that shared-backbone co-evolution strictly dominates disjoint parameter approaches.

Approach. Augment SkillsVote’s lifecycle with a **reflection layer** inspired by RubricEM. After each task, in addition to attribution-based skill evolution, generate a reflection that distills domain-specific adaptation lessons: “this skill worked for terminal tasks but needed X adjustment for SWE tasks,” “the recommendation agent over-selected skills for repository Y—next time, prefer narrower coverage.” Store reflections in a bank indexed by task domain and skill. On subsequent tasks, retrieve relevant reflections to condition both the recommendation agent (adapting search strategy) and the evolution agent (adjusting what constitutes reusable evidence in this domain). The windowed curriculum from RubricEM ($K=3$ step separation) provides temporal structure for building within-domain and cross-domain reflection banks. This transforms offline transfer from a static library copy to a reflection-guided domain adaptation process.

Feasibility. High. RubricEM demonstrates the reflection meta-policy and bank architecture; SkillsVote provides the attribution infrastructure for generating grounded reflections. The main design question is whether reflections should be stored alongside skills (as metadata) or in a separate bank (enabling cross-skill reflection retrieval).

3 Conclusion

SkillsVote makes a systems-level contribution to agent skill ecosystems by introducing attribution-gated lifecycle governance. The 11-category attribution taxonomy—decomposing execution outcomes into skill-linked, responsibility-aware subtasks—addresses the fundamental problem of determining *why* an agent succeeded or failed before allowing evidence to modify persistent skills. The strongest findings are that offline-evolved libraries transfer to unseen tasks with decoupled source-target performance curves, and that recommendation and evolution play complementary roles (recommendation controls negative transfer while evolution creates reusable knowledge). The framework opens rich directions for combining RL-trained attribution with structured taxonomies, applying optimization-geometry tools to library compression, and integrating reflection meta-policies for guided cross-domain transfer.

References

- [1] Liu, H., Yang, H., Jiang, T., Tang, B., Xiong, F., Li, Z. “SkillsVote: Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution.” *arXiv preprint arXiv:2605.18401*, 2026.
- [2] Authors. “SkillOS: Learning Skill Curation for Self-Evolving Agents.” *arXiv preprint arXiv:2605.06614*, 2026.
- [3] Authors. “Memanto: Typed Semantic Memory with Information-Theoretic Retrieval for Long-Horizon Agents.” *arXiv preprint arXiv:2604.22085*, 2026.
- [4] Li, G., et al. “RubricEM: Meta-RL with Rubric-guided Policy Decomposition beyond Verifiable Rewards.” *arXiv preprint arXiv:2605.10899*, 2026.
- [5] Cai, Y., et al. “Learning to Foresee: Unveiling the Unlocking Efficiency of On-Policy Distillation.” *arXiv preprint arXiv:2605.11739*, 2026.